

Python – Matplotlib

Introduction to Matplotlib

Matplotlib is a Python library used for data visualization. It helps us convert numerical data into meaningful graphs and plots, making patterns, trends, and comparisons easy to understand. It is one of the most fundamental libraries in Python and is widely used in data science, machine learning, and scientific computing.

Key reasons to use Matplotlib:

- Easy to learn for beginners
- Highly customizable plots
- Works well with NumPy and Pandas
- Produces publication-quality figures

What is matplotlib.pyplot?

matplotlib.pyplot is a sub-module of Matplotlib that provides a MATLAB-like interface for plotting. It allows us to create plots step by step using simple commands. We usually import it with an alias to reduce typing.

Key points:

- Acts as the main plotting interface
- Controls figure creation, axes, labels, and styles

Example:

```
import matplotlib.pyplot as plt
```

Types of Plots Available in Matplotlib

Matplotlib supports many types of plots for different data visualization needs.

1. Line Plot

Used to show trends over time.

```
plt.plot([1, 2, 3], [5, 10, 15])
```

```
plt.show()
```

2. Bar Plot

Used to compare quantities across categories.

```
plt.bar(["A", "B", "C"], [10, 20, 15])  
plt.show()
```

3. Histogram

Used to show data distribution.

```
plt.hist([2, 4, 4, 6, 8, 8, 10], bins=4)  
plt.show()
```

4. Scatter Plot

Used to show relationship between two variables.

```
plt.scatter([1, 2, 3, 4], [10, 15, 13, 18])  
plt.show()
```

5. Pie Chart

Used to show proportions.

```
plt.pie([40, 30, 20, 10], labels=["A", "B", "C", "D"])  
plt.show()
```

6. Box Plot

Used to show data spread and outliers.

```
plt.boxplot([5, 7, 8, 9, 10, 15])  
plt.show()
```

What is plt.show()?

plt.show() is used to display the final plot on the screen.

In Python scripts, plots will not appear unless plt.show() is called.

Important notes:

- It opens the plot window
- Should usually be the last line of plotting code
- In notebooks, plots may appear automatically, but `plt.show()` is still good practice

Example:

```
plt.plot([1, 2, 3], [10, 20, 30])  
plt.show()
```

Interacting with a Plot

Once a plot window opens, Matplotlib automatically provides interactive tools.

You can:

- Zoom in and zoom out
- Pan across the graph
- Go back and forward through plot views
- Reset to the original view
- Save the plot as an image (PNG, JPG, etc.)

These features help in exploring data visually without writing extra code.

Creating a Basic Line Plot

A line plot is used to show change over time or continuous data.

Example:

```
days = [1, 2, 3, 4, 5]  
temperature = [22, 24, 19, 23, 25]  
plt.plot(days, temperature)  
plt.show()
```

This plot shows how temperature changes across days.

Adding X and Y Labels and Title

Labels and titles make a plot understandable and meaningful.

Why labels are important:

- X-axis label explains the horizontal values
- Y-axis label explains the vertical values
- Title summarizes what the plot represents

Example:

```
plt.plot(days, temperature)
plt.xlabel("Day")
plt.ylabel("Temperature (°C)")
plt.title("Daily Temperature Report")
plt.show()
```

Plotting Multiple Lines on One Graph

Multiple lines are used to compare datasets on the same axes.

Example:

```
week = [1, 2, 3, 4, 5]
city_a = [30, 32, 31, 29, 28]
city_b = [25, 27, 26, 24, 23]

plt.plot(week, city_a, label="City A")
plt.plot(week, city_b, label="City B")
plt.xlabel("Day")
plt.ylabel("Temperature")
plt.title("Temperature Comparison")
plt.legend()
plt.show()
```

Best practice:

```
plt.plot(week, city_a, label="City A")  
plt.plot(week, city_b, label="City B")  
plt.legend()
```

Using Format Strings

Format strings allow us to define color, marker, and line style in one shortcut.

Components:

- Color (r, g, b, etc.)
- Marker (o, ^, s, etc.)
- Line style (--, :, -)

Example:

```
plt.plot(week, city_a, 'bo--', label="City A")  
plt.plot(week, city_b, 'rs:', label="City B")  
plt.legend()  
plt.show()
```

Using Color and Line Style Arguments

Instead of format strings, we can explicitly define styles using arguments. This improves readability and flexibility.

Example:

```
plt.plot(week, city_a, color='purple', linestyle='--', label="City A")  
plt.plot(week, city_b, color='orange', linestyle='-.', label="City B")  
plt.legend()  
plt.show()
```

Line Width and Layout Improvements

Increasing line width makes important lines stand out.
Layout tools improve readability.

Common enhancements:

- linewidth for bold lines
- plt.grid(True) for reference lines
- plt.tight_layout() to avoid overlapping text

Example:

```
plt.plot(week, city_a, linewidth=3, label="City A")  
plt.plot(week, city_b, linewidth=2, label="City B")  
plt.grid(True)  
plt.legend()  
plt.tight_layout()  
plt.show()
```

Using Styles in Matplotlib

Styles apply predefined themes to plots, changing colors and backgrounds.

Benefits:

- Professional-looking plots
- Consistent design
- Saves time on manual styling

Example:

```
plt.style.use("ggplot")  
plt.plot(week, city_a, label="City A")  
plt.legend()  
plt.show()
```

XKCD Comic Style

The XKCD style gives plots a hand-drawn, comic-like appearance.

Example:

with `plt.xkcd()`:

```
plt.plot(week, city_a, label="City A")  
plt.plot(week, city_b, label="City B")  
plt.title("Funny Temperature Battle")  
plt.legend()  
plt.show()
```

Markers, Colors, and Line Styles in Matplotlib

Common Markers

Markers represent individual data points.

Marker	Meaning
o	Circle
.	Point
^	Triangle up
v	Triangle down
s	Square
*	Star
x	Cross
+	Plus

Example:

```
plt.plot([1, 2, 3], [4, 6, 8], marker='o')  
plt.show()
```

Common Colors

Colors define the appearance of lines or markers.

Code	Color
b	Blue
g	Green
r	Red
c	Cyan
m	Magenta
y	Yellow
k	Black

You can also use color names like "orange", "purple", "brown".

Line Styles

Line styles control how lines are drawn.

Style	Meaning
-	Solid line
--	Dashed line
-.	Dash-dot line
:	Dotted line

Example:

```
plt.plot([1, 2, 3], [3, 6, 9], linestyle=':')  
plt.show()
```

Matplotlib Bar Charts

What is a Bar Chart?

A **bar chart** is used to compare **quantities across different categories**.

Each bar represents a category, and the **height (or length)** of the bar shows its value.

Bar charts are:

- Easy to understand
- Excellent for comparisons
- Commonly used in reports, dashboards, and presentations

Types of Bar Charts in Matplotlib

Matplotlib mainly supports:

- **Vertical bar charts** → `plt.bar()`
- **Horizontal bar charts** → `plt.barh()`
- **Grouped (side-by-side) bar charts**

Vertical Bar Chart (`plt.bar()`)

A vertical bar chart displays categories on the **x-axis** and values on the **y-axis**.

Example Data:

```
subjects = ["Maths", "Physics", "Chemistry", "Biology"]
```

```
marks = [78, 85, 69, 90]
```

Basic Vertical Bar Chart:

```
import matplotlib.pyplot as plt
```

```
plt.bar(subjects, marks)
```

```
plt.xlabel("Subjects")
```

```
plt.ylabel("Marks")
```

```
plt.title("Student Marks in Different Subjects")
```

```
plt.show()
```

Key Points:

- `plt.bar(x, y)` creates vertical bars
- Categories are placed on the x-axis
- Values determine bar height

Customizing Bar Width

The width parameter controls the thickness of bars.

Important Notes:

- Default width is 0.8
- Value range is usually between 0 and 1
- Larger values may cause bars to overlap

Example:

```
plt.bar(subjects, marks, width=0.5)
plt.show()
```

Grouped (Side-by-Side) Bar Charts

Grouped bar charts are used to **compare multiple datasets across the same categories**.

Example Scenario:

Comparing marks of **three students** in the same subjects.

Data:

```
import numpy as np
```

```
subjects = ["Maths", "Physics", "Chemistry", "Biology"]
```

```
student1 = [78, 85, 69, 90]
```

```
student2 = [82, 80, 75, 88]
```

```
student3 = [70, 78, 72, 85]
```

```
x = np.arange(len(subjects))
```

width = 0.25

Plotting Side-by-Side Bars:

```
plt.bar(x - width, student1, width=width, label="Student 1")
```

```
plt.bar(x, student2, width=width, label="Student 2")
```

```
plt.bar(x + width, student3, width=width, label="Student 3")
```

```
plt.xlabel("Subjects")
```

```
plt.ylabel("Marks")
```

```
plt.title("Marks Comparison")
```

```
plt.xticks(x, subjects)
```

```
plt.legend()
```

```
plt.tight_layout()
```

```
plt.show()
```

Why Use plt.xticks()?

By default, Matplotlib uses numeric positions like 0, 1, 2, ... on the x-axis.

Purpose of plt.xticks():

- Replaces numeric positions with **meaningful labels**
- Improves readability of the chart

Example:

```
plt.xticks(x, subjects)
```

Without this, categories may appear as numbers instead of names.

Horizontal Bar Charts (plt.barh())

Horizontal bar charts display:

- Categories on the **y-axis**
- Values on the **x-axis**

When to Use Horizontal Bars:

- When category names are long
- When you want left-to-right comparison
- When many categories exist

Example Data:

```
departments = ["HR", "Finance", "IT", "Marketing"]
```

```
employees = [12, 18, 25, 15]
```

Horizontal Bar Chart:

```
plt.barh(departments, employees, color="lightgreen")
```

```
plt.xlabel("Number of Employees")
```

```
plt.title("Employees per Department")
```

```
plt.tight_layout()
```

```
plt.show()
```

Difference Between bar() and barh()

Function	Categories	Values
plt.bar()	x-axis	y-axis
plt.barh()	y-axis	x-axis

Adding Value Labels on Bars (plt.text())

Value labels display the **exact numbers** on or near the bars, making charts more informative.

Why Use Value Labels:

- Improves clarity
- Helpful in presentations
- Eliminates guesswork for viewers

Example:

```
products = ["Laptop", "Tablet", "Mobile"]
```

```
sales = [150, 120, 200]
```

```
plt.bar(products, sales, color="skyblue")
```

```
for i in range(len(products)):
```

```
    plt.text(i, sales[i] + 5, str(sales[i]), ha='center')
```

```
plt.ylabel("Units Sold")
```

```
plt.title("Product Sales Report")
```

```
plt.tight_layout()
```

```
plt.show()
```

Tips:

- Use ha='center' to center the text
- Add a small offset (+5) to avoid overlapping with bars

Common Customization Options for Bar Charts

- color → change bar color
- width → adjust bar thickness
- label → used with legends
- plt.grid(axis='y') → improves readability
- plt.tight_layout() → avoids overlapping elements

Matplotlib Pie Charts

What is a Pie Chart?

A **pie chart** is used to represent **part-to-whole relationships**.

Each slice of the pie shows how much a category contributes to the total.

Characteristics:

- The entire pie represents **100%**
- Each slice represents a proportion of the total
- Best used when the number of categories is **small**

When to Use Pie Charts:

- To show percentage or share distribution
- When comparing proportions, not exact values
- For presentations and summary reports

When to Avoid Pie Charts:

- Too many categories (becomes cluttered)
- When precise comparison is required (use bar charts instead)

Setting a Style for Pie Charts

Matplotlib allows you to apply predefined styles to improve appearance.

Example:

```
import matplotlib.pyplot as plt
```

Benefits of Styles:

- Better color contrast
- Cleaner background
- Professional appearance

Basic Pie Chart

Example Data:

```
labels = ["Rent", "Food", "Transport", "Entertainment"]
```

```
expenses = [40, 25, 20, 15]
```

Basic Pie Chart Code:

```
plt.pie(expenses, labels=labels)  
plt.title("Monthly Expense Distribution")  
plt.show()
```

Key Points:

- `plt.pie()` creates the pie chart
- labels assign names to slices
- Values are automatically converted to percentages

Custom Colors in Pie Charts

You can assign custom colors using **color names or hex codes**.

Example:

```
colors = ["#ff9999", "#66b3ff", "#99ff99", "#ffcc99"]  
  
plt.pie(expenses, labels=labels, colors=colors)  
plt.title("Expense Breakdown with Custom Colors")  
plt.show()
```

Benefits:

- Improves visual clarity
- Helps differentiate slices
- Matches presentation themes

Styling Slices Using `wedgeprops`

`wedgeprops` allows you to customize the appearance of pie slices.

Common Properties:

- `edgecolor` → color of slice borders
- `linewidth` → thickness of borders

- linestyle → solid, dashed, dotted lines

Example:

```
plt.pie(  
    expenses,  
    labels=labels,  
    colors=colors,  
    wedgeprops={'edgecolor': 'black', 'linewidth': 2, 'linestyle': '--'}  
)  
plt.title("Styled Pie Chart")  
plt.show()
```

Highlighting a Slice Using explode

The explode parameter is used to **separate one or more slices** from the pie.

Why Use Explode:

- Draw attention to an important category
- Highlight the largest or smallest share

Example:

```
explode = [0.1, 0, 0, 0] # Highlight Rent
```

```
plt.pie(  
    expenses,  
    labels=labels,  
    explode=explode,  
    colors=colors  
)  
plt.title("Highlighted Expense Category")  
plt.show()
```

Notes:

- Values represent how far the slice moves outward
- Use small values like 0.05 or 0.1

Adding Shadows for 3D Effect

Shadows add a slight **3D-like visual effect**.

Example:

```
plt.pie(  
    expenses,  
    labels=labels,  
    explode=explode,  
    colors=colors,  
    shadow=True  
)  
plt.title("Pie Chart with Shadow Effect")  
plt.show()
```

Tip:

Use shadows sparingly to avoid visual overload.

Rotating the Pie Using startangle

startangle rotates the pie chart for better alignment.

Example:

```
plt.pie(  
    expenses,  
    labels=labels,  
    colors=colors,  
    startangle=140  
)  
plt.title("Rotated Pie Chart")
```

```
plt.show()
```

Why Use startangle?

- Improves readability
- Helps align important slices at the top

Displaying Percentages with autopct

The autopct parameter displays **percentage values on slices**.

Example:

```
plt.pie(  
    expenses,  
    labels=labels,  
    autopct='%.1f%%',  
    startangle=140  
)  
plt.title("Expense Percentage Share")  
plt.show()
```

Common Format Options:

- `%.0f%%` → whole numbers
- `%.1f%%` → one decimal place
- `%.2f%%` → two decimal places

Combining Multiple Pie Chart Features

You can combine explode, colors, shadow, and percentages together.

Example:

```
explode = [0.1, 0, 0, 0]
```

```
plt.pie(  
    expenses,
```

```
labels=labels,  
colors=colors,  
explode=explode,  
autopct='%1f%%',  
shadow=True,  
startangle=140  
)  
plt.title("Complete Pie Chart Example")  
plt.show()
```

Stack Plots in Matplotlib

Understanding the Idea Before Stack Plots

Before learning stack plots, it helps to understand how data can represent **parts of a whole**.

A **pie chart** shows how different components contribute to a whole **at a single point in time**.

However, when we want to see **how those contributions change over time**, a pie chart is no longer sufficient.

This is where **stack plots** are useful.

What is a Stack Plot?

A **stack plot** is a type of **area chart** where multiple data series are plotted on top of each other along the y-axis.

Each colored area represents one category, and the total height at any point represents the **combined total** of all categories.

Key Characteristics:

- Displays **multiple datasets together**
- Shows both **individual contribution** and **overall trend**
- Useful for time-series data

When to Use Stack Plots

Stack plots are especially useful when:

- You want to track how different parts contribute to a total **over time**
- The total value itself is important
- You want a smoother alternative to grouped bar charts

Common Use Cases:

- Daily energy usage from different sources
- Monthly expenses split by category
- Website traffic from different platforms over time
- Work hours distributed among tasks during a project

Basic Syntax of plt.stackplot()

`plt.stackplot(x, y1, y2, y3, ...)`

Where:

- `x` → x-axis values (usually time)
- `y1, y2, y3...` → different data series to be stacked

Stack Plot Example

Scenario:

Tracking how a person spends their **daily working hours** across a week.

Data:

```
import matplotlib.pyplot as plt
```

```
days = [1, 2, 3, 4, 5]
```

```
coding = [4, 5, 6, 5, 4]
```

```
meetings = [2, 1, 1, 2, 3]
```

```
learning = [2, 2, 1, 1, 1]
```

```
breaks = [1, 1, 2, 1, 1]
```

```
labels = ["Coding", "Meetings", "Learning", "Breaks"]
```

```
colors = ["skyblue", "lightgreen", "gold", "lightcoral"]
```

Creating the Stack Plot:

```
plt.figure(figsize=(10, 6))
```

```
plt.stackplot(
```

```
    days,
```

```
    coding,
```

```
    meetings,
```

```
    learning,
```

```
    breaks,
```

```
    labels=labels,
```

```
    colors=colors,
```

```
    alpha=0.8
```

```
)
```

```
plt.legend(loc="upper left")
```

```
plt.xlabel("Day")
```

```
plt.ylabel("Hours")
```

```
plt.title("Weekly Work Activity Distribution")
```

```
plt.grid(True)
```

```
plt.show()
```

Interpreting a Stack Plot

From the stack plot, we can observe:

- How each activity changes over days
- Which activity takes the most time

- How the **total working hours** vary day by day
- Relative contribution of each category

The top edge of the stack represents the **total value** at that point.

Adding Transparency with alpha

The alpha parameter controls transparency.

Values:

- alpha = 0 → fully transparent
- alpha = 1 → fully opaque

Why Use alpha?

- Makes overlapping areas easier to distinguish
- Improves readability when many stacks exist

Example:

```
plt.stackplot(days, coding, meetings, alpha=0.7)
```

Using Labels and Legend

Labels are passed using the labels parameter and displayed using `plt.legend()`.

Example:

```
plt.legend(loc="upper left")
```

Common Legend Locations:

- "upper left"
- "upper right"
- "lower left"
- "best"

Grid and Layout Improvements

Adding a grid improves readability and helps track values across axes.

Example:

```
plt.grid(True)
```

You can also adjust figure size using:

```
plt.figure(figsize=(width, height))
```

Key Parameters of `plt.stackplot()`

Parameter	Description
x	X-axis values (time, days, years, etc.)
*args	Multiple y-values to be stacked
labels	Labels for legend
colors	List of colors for each stack
alpha	Transparency level (0 to 1)

Histograms in Matplotlib

What is a Histogram?

A **histogram** is a graphical representation of the **distribution of numerical data**.

It groups continuous data into intervals called **bins** and displays how many data points fall into each bin.

Key Idea:

- **X-axis** → value ranges (bins)
- **Y-axis** → frequency (count of values)

Histograms help us understand **data spread, concentration, skewness, and outliers**.

Why and When to Use Histograms

Use histograms when:

- You want to study the **distribution** of numeric data
- Data is continuous (not categorical)

- You want to identify patterns like clustering or gaps

Common Use Cases:

- Exam score distribution
- Salary range analysis
- Website session duration
- Sensor or measurement data analysis

Difference Between Histogram and Bar Chart

Feature	Histogram	Bar Chart
Data type	Continuous	Categorical
Bars	Touch each other	Usually separated
Purpose	Distribution	Comparison
X-axis	Ranges (bins)	Categories

Creating a Basic Histogram

Example Data:

```
import matplotlib.pyplot as plt
```

```
scores = [45, 55, 60, 70, 72, 75, 80, 85, 90, 95, 88, 76, 67, 59, 62]
```

Basic Histogram:

```
plt.hist(scores)
plt.xlabel("Score Range")
plt.ylabel("Number of Students")
plt.title("Exam Score Distribution")
plt.show()
```

Understanding the bins Argument

The bins parameter controls how data is grouped.

1. Integer Bins (Equal Width)

Defines the number of equal-width bins.

```
plt.hist(scores, bins=5)
```

2. Custom Bin Ranges

Defines exact bin edges.

```
bins = [40, 50, 60, 70, 80, 90, 100]
```

```
plt.hist(scores, bins=bins)
```

Why Custom Bins?

- Better control over data grouping
- More meaningful ranges
- Useful for age, salary, or score brackets

Improving Visibility with edgecolor

The edgecolor parameter adds borders around histogram bars.

Example:

```
plt.hist(scores, bins=5, edgecolor='black')
```

Benefits:

- Bars are clearly separated
- Easier to read overlapping regions
- Improves presentation quality

Customizing Histogram Appearance

You can customize histograms using:

- color → bar color
- alpha → transparency
- linestyle → grid or edges

Example:

```
plt.hist(scores, bins=5, color='skyblue', edgecolor='black', alpha=0.8)

plt.grid(True, linestyle='--', alpha=0.5)

plt.show()
```

Example: Distribution of Daily Study Hours

Data:

```
hours = [1, 2, 2, 3, 4, 5, 3, 2, 6, 4, 3, 5, 2, 4, 3, 1, 2, 5, 6, 4]

bins = [0, 1, 2, 3, 4, 5, 6, 7]
```

Histogram:

```
plt.hist(hours, bins=bins, edgecolor='black')

plt.xlabel("Study Hours per Day")

plt.ylabel("Number of Students")

plt.title("Daily Study Hours Distribution")

plt.grid(True, linestyle='--', alpha=0.5)

plt.show()
```

Adding a Vertical Line with axvline()

axvline() is used to draw a **vertical reference line** at a specific x-value.

Common Uses:

- Marking average (mean)
- Showing pass/fail cutoff
- Highlighting thresholds

Example:

```
import numpy as np

plt.hist(hours, bins=bins, edgecolor='black')

plt.axvline(
```

```
np.mean(hours),  
color='red',  
linestyle='--',  
linewidth=2,  
label='Average Study Time'  
)  
plt.legend()  
plt.title("Study Hours with Average Line")  
plt.show()
```

Interpreting a Histogram

From a histogram, you can identify:

- Most common value ranges
- Data spread (wide or narrow)
- Skewness (left or right)
- Presence of outliers

Scatter Plots in Matplotlib

What is a Scatter Plot?

A **scatter plot** is used to show the **relationship between two numerical variables**.

Each point on the plot represents a single observation with:

- An **x-value** (independent variable)
- A **y-value** (dependent variable)

Scatter plots are especially useful for identifying:

- Trends and patterns
- Correlation (positive, negative, or none)
- Clusters and outliers

When to Use Scatter Plots

Use scatter plots when:

- Both variables are **numeric**
- You want to explore relationships or correlations
- You want to detect unusual data points

Common Use Cases:

- Hours of practice vs performance
- Advertising spend vs sales
- Temperature vs electricity usage
- Height vs weight analysis

Creating a Basic Scatter Plot

Example Data:

```
import matplotlib.pyplot as plt
```

```
practice_hours = [1, 2, 3, 4, 5, 6, 7, 8]
```

```
test_scores = [42, 48, 52, 58, 63, 70, 78, 88]
```

Basic Scatter Plot:

```
plt.scatter(practice_hours, test_scores)
```

```
plt.title("Practice Hours vs Test Scores")
```

```
plt.xlabel("Practice Hours")
```

```
plt.ylabel("Test Scores")
```

```
plt.grid(True)
```

```
plt.show()
```

Interpretation:

An upward pattern suggests a **positive relationship** between practice time and performance.

Customizing Point Color and Size

You can change the **appearance of points** to convey more information.

Common Parameters:

- `c` → color of points
- `s` → size of points
- `alpha` → transparency

Example:

```
sizes = [score * 1.5 for score in test_scores]
```

```
colors = ['red' if score < 60 else 'green' for score in test_scores]
```

```
plt.scatter(practice_hours, test_scores, s=sizes, c=colors)
```

```
plt.title("Scatter Plot with Size and Color Encoding")
```

```
plt.xlabel("Practice Hours")
```

```
plt.ylabel("Test Scores")
```

```
plt.grid(True)
```

```
plt.show()
```

Meaning:

- Larger points represent higher scores
- Red points indicate lower performance
- Green points indicate better performance

Using Colormaps in Scatter Plots

Colormaps allow you to apply **gradient coloring** based on numerical values.

Why Use Colormaps?

- Shows smooth variation
- Useful for large datasets
- Makes trends easier to spot

Example:

```
import numpy as np
```

```
score_array = np.array(test_scores)
```

```
plt.scatter(practice_hours, test_scores, c=score_array, cmap='plasma')
```

```
plt.colorbar(label="Score Level")
```

```
plt.title("Scatter Plot with Colormap")
```

```
plt.xlabel("Practice Hours")
```

```
plt.ylabel("Test Scores")
```

```
plt.grid(True)
```

```
plt.show()
```

Notes:

- cmap defines the color gradient
- plt.colorbar() explains the color scale

Adding Annotations to Points

Annotations allow you to **label individual data points**.

When to Use Annotations:

- Small datasets
- Highlighting specific observations
- Presentations or reports

Example:

```
plt.scatter(practice_hours, test_scores)
```

```
for i in range(len(practice_hours)):
```

```
    plt.annotate(  
        f"Point {i+1}",
```

```
(practice_hours[i], test_scores[i])  
)
```

```
plt.title("Scatter Plot with Annotations")  
plt.xlabel("Practice Hours")  
plt.ylabel("Test Scores")  
plt.grid(True)  
plt.show()
```

Plotting Multiple Groups in One Scatter Plot

Multiple datasets can be shown on the same scatter plot for comparison.

Example Scenario:

Comparing **two training programs**.

Data:

```
program_a_hours = [2, 4, 6, 8]
```

```
program_a_scores = [50, 60, 70, 85]
```

```
program_b_hours = [1, 3, 5, 7, 9]
```

```
program_b_scores = [45, 55, 65, 75, 90]
```

Scatter Plot with Multiple Groups:

```
plt.scatter(program_a_hours, program_a_scores, label="Program A", color="blue")
```

```
plt.scatter(program_b_hours, program_b_scores, label="Program B", color="orange")
```

```
plt.title("Training Program Comparison")  
plt.xlabel("Practice Hours")  
plt.ylabel("Scores")  
plt.legend()  
plt.grid(True)
```

plt.show()

Common Scatter Plot Parameters

Parameter	Description
c	Color of points
s	Size of points
alpha	Transparency
cmap	Colormap
label	Legend label

Scatter Plot vs Line Plot

Feature	Scatter Plot	Line Plot
Data connection	No	Yes
Best for	Relationship	Trend over time
Order matters	No	Yes

Subplots in Matplotlib

What are Subplots?

Subplots allow you to display **multiple plots inside a single figure**, arranged in rows and columns. They are especially useful when you want to **compare related datasets** or show **different views of the same data** at once.

Instead of opening many separate figures, subplots organize visualizations neatly in one window.

Why Use Subplots?

Subplots are helpful when:

- Comparing multiple datasets side by side
- Showing different transformations of the same data

- Building dashboards or reports
- Saving multiple related plots as one image

Basic Subplot Using plt.subplot()

The syntax of plt.subplot() is:

```
plt.subplot(rows, columns, plot_number)
```

Example: 1 Row × 2 Columns

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]
```

```
y_linear = [v + 2 for v in x]
```

```
y_cube = [v ** 3 for v in x]
```

```
plt.subplot(1, 2, 1)
```

```
plt.plot(x, y_linear)
```

```
plt.title("x + 2")
```

```
plt.subplot(1, 2, 2)
```

```
plt.plot(x, y_cube)
```

```
plt.title("x cubed")
```

```
plt.tight_layout()
```

```
plt.show()
```

Explanation:

- 1, 2 → layout (1 row, 2 columns)
- 1 and 2 → position of the subplot
- Each plt.subplot() call activates a specific plot area

Creating a 2 × 2 Grid of Subplots

A grid layout allows you to show more plots at once.

Example:

```
y_square = [v ** 2 for v in x]
```

```
y_sqrt = [v ** 0.5 for v in x]
```

```
y_reverse = [8 - v for v in x]
```

```
plt.figure(figsize=(8, 6))
```

```
plt.subplot(2, 2, 1)
```

```
plt.plot(x, y_linear)
```

```
plt.title("x + 2")
```

```
plt.subplot(2, 2, 2)
```

```
plt.plot(x, y_square)
```

```
plt.title("x squared")
```

```
plt.subplot(2, 2, 3)
```

```
plt.plot(x, y_sqrt)
```

```
plt.title("Square Root of x")
```

```
plt.subplot(2, 2, 4)
```

```
plt.plot(x, y_reverse)
```

```
plt.title("8 - x")
```

```
plt.tight_layout()
```

```
plt.show()
```

Key Point:

- 2×2 layout creates **four plots**
- `plt.tight_layout()` prevents overlapping titles and labels

Using `plt.subplots()` (Recommended Approach)

`plt.subplots()` is a **cleaner and more powerful** way to work with subplots.

Syntax:

```
fig, axs = plt.subplots(rows, cols)
```

- `fig` → entire figure
- `axs` → array of axes (individual plots)

Example: 1 Row × 2 Columns

```
fig, axs = plt.subplots(1, 2, figsize=(10, 4))
```

```
axs[0].plot(x, y_linear)
```

```
axs[0].set_title("x + 2")
```

```
axs[1].plot(x, y_cube)
```

```
axs[1].set_title("x cubed")
```

```
fig.suptitle("Function Comparison", fontsize=14)
```

```
fig.tight_layout()
```

```
fig.subplots_adjust(top=0.85)
```

```
plt.show()
```

Why `plt.subplots()` is Better:

- Cleaner code
- Easier to loop over plots
- Better control over layout
- Preferred for advanced visualizations

Figure (fig) vs Axes (axs)

Object	Purpose
fig	Controls the entire figure (title, size, save)
axs	Controls individual subplots

Example:

```
fig.savefig("comparison_plots.png")
```

Looping Over Subplots

Looping is useful when plotting **multiple datasets dynamically**.

Example: 2 × 2 Subplots Using Loop

```
fig, axs = plt.subplots(2, 2, figsize=(8, 6))

ys = [y_linear, y_square, y_sqrt, y_reverse]
titles = ["x + 2", "x squared", "sqrt(x)", "8 - x"]

for i in range(2):
    for j in range(2):
        index = i * 2 + j
        axs[i, j].plot(x, ys[index])
        axs[i, j].set_title(titles[index])

plt.tight_layout()
plt.show()
```

Advantage:

- Reduces repeated code
- Scales easily for larger grids

Saving Subplots as an Image

You can save the entire figure using `fig.savefig()`.

Example:

```
fig.savefig("multi_plot_figure.png")
```

Common Subplot Layouts

Layout	Use Case
1 × 2	Side-by-side comparison
2 × 2	Multiple transformations
3 × 1	Time-based trends
Grid	Dashboards

